Laboratório de Sistemas Operacionais – sched_profiler

Calvin Silva, João Medeiros, Pedro Jucewicz · PUCRS · novembro de 2026

1. Implementação

Laço de cada thread. A thread converte seu índice em letra (`'A' + id`) e repete: `sem_wait`; se `ptr >= buffer_end`, faz `sem_post` e sai; senão escreve a letra, avança o ponteiro e faz `sem_post`. O teste de fim fica dentro da seção crítica: fora dela, duas threads poderiam ver a última posição livre e uma escreveria além do buffer. O `sem_post` antes do `break` evita que a thread termine segurando o semáforo, o que deixaria as demais bloqueadas para sempre.

Política e largada. As threads são criadas com a política padrão e só depois recebem a política pedida (`pthread_setschedparam`, prioridade 1 para FIFO/RR e 0 para IDLE/LOW_IDLE). Para que nenhuma thread escreva nesse intervalo, todas esperam numa barreira (`pthread_barrier_t`) com N+1 participantes: ela só abre quando o main também chega, e ele só chega depois de configurar todas. Sem ela, a thread A começaria a escrever enquanto as outras ainda nem existem, e o buffer refletiria a ordem de criação, não o escalonador. Se o kernel recusar a política, o programa aborta, para não medir com a política errada. `SCHED_LOW_IDLE` é definida no código com o valor 7, o mesmo do kernel modificado, pois a glibc não a conhece.

Pós-processamento. Uma única passada no buffer conta, para cada letra, as posições escritas e os blocos (uma troca de letra inicia um bloco novo, que é impresso no resumo). O número de blocos é o que o programa chama de "escalonamentos".

2. Ambiente e parâmetros

QEMU i386 com **1 núcleo** (`-smp cpus=1`), kernel Linux 4.13.9 com a política `SCHED_LOW_IDLE`, distribuição gerada com Buildroot. Parâmetros: **buffer de 5.000.000 posições e 4 threads** (cerca de 1 a 2,5 s de execução no emulador, tempo suficiente para várias fatias de 100 ms do RR). Cada política foi executada uma vez, com o trace capturado durante a execução:

```
trace-cmd record -o rr.dat -e sched:sched_switch \
  sh -c "sched_profiler 5000000 4 SCHED_RR > rr.txt"
```

3. Saída de exemplo (SCHED_RR, buffer truncado)

```
buffer = 5000000, threads = 4, politica = SCHED_RR
buffer alocado em 0xb7909010
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
ABCDABCDABCDABCD

Escalonamentos:
A = 4
B = 4
C = 4
D = 4

Escritas no buffer:
A = 2411410 (48.2%)
B = 1149973 (23.0%)
C = 425256 (8.5%)
D = 1013361 (20.3%)
```

4. Resultados

Política	Resumo (início)	Escalon. A/B/C/D	Escritas (%) A/B/C/D	Troca de thread
FIFO	A	1/0/0/0	100/0/0/0	nenhuma
RR	ABCDAB	4/4/4/4	48/23/9/20	~100 ms, ordem fixa
IDLE	CDBACD	103/92/98/98	27/21/27/25	~4 ms, aleatória
LOW_IDLE	BCDABC	109/114/113/115	24/26/26/24	~4 ms, aleatória

"Escalonamentos" conta os blocos de cada letra no buffer: fatias consecutivas da mesma thread formam um único bloco. As entradas reais na CPU aparecem no trace.

5. Análise

SCHED_FIFO. As threads FIFO têm prioridade sobre o main (normal): cada uma, ao receber a política, toma a CPU, chega à barreira e dorme. Assim elas chegam na ordem A, B, C, D. Quando o main libera a barreira, A está na frente da fila e, como a FIFO não tem fatia de tempo e A nunca bloqueia (o semáforo está sempre livre), executa sozinha por cerca de 1,1 s até encher o buffer (resumo **A**, 100% das escritas). Nesta execução não houve pausa do RT throttling (limite de 950 ms por período de 1 s). Depois, B, C e D executam por microssegundos, encontram o buffer cheio e terminam (estado x).

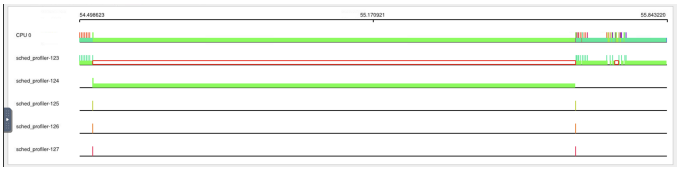
SCHED_RR. A largada é igual à da FIFO, mas cada thread recebe uma fatia de ~100 ms e volta ao fim da fila (trocas com estado R), gerando a ordem circular A, B, C, D. O RR é justo nas fatias, mas não no trabalho: quando a fatia termina com a thread dentro da seção crítica, as demais tentam o semáforo, dormem (S) e a CPU volta à mesma thread, que ganha uma fatia extra (A teve ~796 ms de CPU e D ~313 ms). Além disso, enquanto há threads acordadas que ainda não reexecutaram, cada sem_post precisa de uma chamada ao kernel (futex), reduzindo a velocidade de escrita; isso explica, provavelmente, C ter escrito só 8,5% do buffer com ~400 ms de CPU. O RT throttling gerou duas pausas de ~50 ms.

SCHED_IDLE e SCHED_LOW_IDLE. Aqui as threads têm prioridade inferior à do main: nenhuma executa durante a criação, o main chega primeiro à barreira e a última thread (D) é quem a libera. Ao acordar, o main preempta D (tarefa normal sempre preempta uma ociosa), entra no pthread_join e dorme. O CFS escolhe sempre a thread com menor vruntime; com peso muito baixo (3 no IDLE, 1 no LOW_IDLE), poucos microssegundos de CPU antes da barreira já decidem quem começa (C no IDLE, B no LOW_IDLE). As trocas ocorrem a cada ~4 ms (tick de 250 Hz), sem ordem fixa, e a CPU é dividida de forma equilibrada (21% a 27% das escritas por thread). Os padrões são semelhantes porque todas as threads usam a mesma política: a SCHED_LOW_IDLE só se diferencia ao competir com tarefas de outras políticas.

Conclusão. As políticas de tempo real (FIFO, RR) passam à frente das tarefas normais e geram poucos blocos longos, diferenciando-se pela fatia de tempo; as políticas do CFS (IDLE, LOW_IDLE) geram centenas de blocos curtos e uma divisão equilibrada da CPU. A quantidade escrita por thread também depende do semáforo, e não só do escalonador.

6. Saídas do programa e visualização no KernelShark

Em cima, o trace (evento sched:sched_switch). Em cada linha de thread: verde = executando, retângulo vermelho = pronta (na fila), linha preta = dormindo; o primeiro PID é o main e os quatro seguintes são A a D. Embaixo, a saída do sched_profiler na mesma execução, como impressa pelo programa (só a linha do buffer foi truncada; o buffer completo está em traces/*.txt).



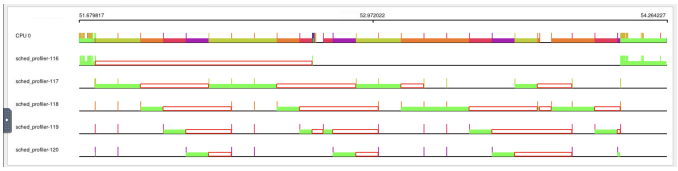
```
buffer = 5000000, threads = 4, politica = SCHED_FIFO
buffer alocado em 0xb7905010
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
A

Escalonamentos:
A = 1
B = 0
C = 0
D = 0

Escritas no buffer:
A = 5000000 (100.0%)
B = 0 (0.0%)
C = 0 (0.0%)
D = 0 (0.0%)
```

Figura 1 – SCHED_FIFO (main = 123, A–D = 124–127): A executa sozinha até encher o buffer; B, C e D só aparecem na largada e ao terminar.



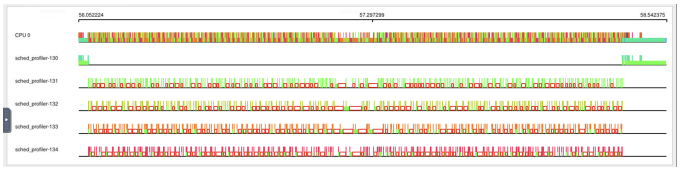
```
buffer = 5000000, threads = 4, politica = SCHED_RR
buffer alocado em 0xb7909010
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
ABCDABCDABCDABCD

Escalonamentos:
A = 4
B = 4
C = 4
D = 4

Escritas no buffer:
A = 2411410 (48.2%)
B = 1149973 (23.0%)
C = 425256 (8.5%)
D = 1013361 (20.3%)
```

Figura 2 – SCHED_RR (main = 116, A–D = 117–120): fatias de ~100 ms em ordem circular fixa.



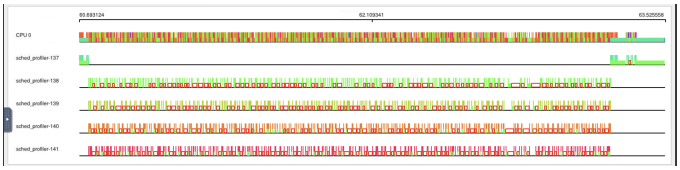
```
buffer = 5000000, threads = 4, politica = SCHED_IDLE
buffer alocado em 0xb7905010
CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC...

Resumo:
CDBACDABCDADBCADCABADCBADACBDACBDACBDBACDBACBDACDBACDBACDABCADBCAD
CBADCBADCBADCBCABDCABDCABCDABCDACBACDBACBDBACDABCDABCDABCDABCDABCA
DBCADCBADCBADCBADCBADCBADCBADCABCDACDCABDACBDCDBACBDADCBADCBDACADB
DACDBDADABCDABDCABDCACBDACABDCABDACBACDABCDABCADBCADBCADBADCABDACD
ACBDADCBDACADBCABDBCABCADBCADBCDABCDABCDABCDBACDBACDBACBDACBDACBDA
CBDACBDCABDCADBCADBACDACBDACBDABCADCBADCABDCBADCBADACBDCABDAC

Escalonamentos:
A = 103
B = 92
C = 98
D = 98

Escritas no buffer:
A = 1325272 (26.5%)
B = 1054884 (21.1%)
C = 1349439 (27.0%)
D = 1270405 (25.4%)
```

Figura 3 – SCHED_IDLE (main = 130, A–D = 131–134): trocas a cada ~4 ms, sem ordem fixa; o main só executa no início e no fim.



```
buffer = 5000000, threads = 4, politica = SCHED_LOW_IDLE
buffer alocado em 0xb78da010
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB...

Resumo:
BCDABCADBCABDCBADCBDACBDACADBCDABCDABCDACBDACBDACBDACBADCBADCBADCB
ADCBADCABDADBCADBACDBCABDBDCABDCBADCBADCBADCBADCABADCBDCACBDAC
BDACBDBADCABDCABDCABDCBDACBCDACBDACBDACDBACDBDACBDACBCDACBDABCDABC
DABCDBACDABCDABCADBCABDCBDADBACABDACBADCBADCBADBADCBCADCABDCABADCB
DCACBADCBADCBACDBACDBACBDABCDACBADBCBDACBDBACADBCADBCADCBADCDBACDB
DACBDACBACDBADBCACDBDACBADCBADCBADCBCDABCDCBDACBDACBDCDBADCBDABCDA
BCDABDCABDCDBACDBCADCBADCBADCBADCBACDBACDBACDBDACBDCBAD

Escalonamentos:
A = 109
B = 114
C = 113
D = 115

Escritas no buffer:
A = 1185103 (23.7%)
B = 1293111 (25.9%)
C = 1307715 (26.2%)
D = 1214071 (24.3%)
```

Figura 4 – SCHED_LOW_IDLE (main = 137, A–D = 138–141): padrão semelhante ao SCHED_IDLE.